SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1001	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	S.ASWINI	**Reg. No.:**	192511077
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:
Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.

- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)

System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical	Provides strong justification for architectural decisions, discusses	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of	Justification is weak or absent; report lacks

Presentation (10%)	trade-offs and future enhancements, and presents a well-organized, professional report.		trade-offs or future scope; report needs improvement.	organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. Analyze System Requirements

1.1 System Overview

The Digital Platform to Crowdsource Societal Challenges is a web-based platform where citizens can report community issues, share ideas, support existing reports, and follow progress. Moderators review submissions, while administrators and authorized partner organizations manage the platform and publish updates.

1.2 System Objectives

- Collect societal challenges in one centralized platform.
- Encourage citizens to participate through comments, votes, and suggestions.
- Help moderators review reports and identify unsuitable or duplicate content.
- Allow authorized partners to communicate progress and resolution updates.
- Provide transparent status tracking and summary reports.

1.3 Functional Requirements Affecting Architecture

ID	Requirement	Architectural Impact
FR1	Register, log in, and manage user profiles	Requires authentication and user management components.
FR2	Submit a challenge with category, description, area, and optional image	Requires input validation, challenge service, database, and file storage.
FR3	Search and filter challenges	Requires searchable data and query handling.
FR4	Vote, comment, and follow challenges	Requires participation services and relationships between users and challenges.
FR5	Moderate content and handle abuse reports	Requires role-based permissions and a moderation queue.
FR6	Track status and publish progress updates	Requires status workflow and update history.
FR7	Notify users of important changes	Requires a notification component or external service.
FR8	Display analytics and administrative reports	Requires aggregation queries and dashboard components.

1.4 Non-Functional Requirements

Quality Attribute	Requirement / Target
Scalability	Support growth in users and submissions by allowing application capacity to be increased.
Security	Use HTTPS, secure password hashing, role-based access, input validation, and protected sessions.
Performance	Common pages should aim to load within 3 seconds under normal expected load.
Reliability	Store reports consistently and back up the database regularly.
Availability	Target 99% monthly availability, excluding planned maintenance, subject to hosting capability.

Maintainability	Separate interface, business logic, and data access so each can be updated independently.
Privacy	Collect only necessary personal information and restrict access to private user data.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Three-Tier Layered Architecture

A Three-Tier Layered Architecture is proposed, with a Presentation Layer, an Application/Business Layer, and a Data Layer. A small API/data-access boundary connects the business logic to storage and external services. The design can be implemented as a modular monolithic application initially, without the operational complexity of multiple microservices.

2.2 Justification

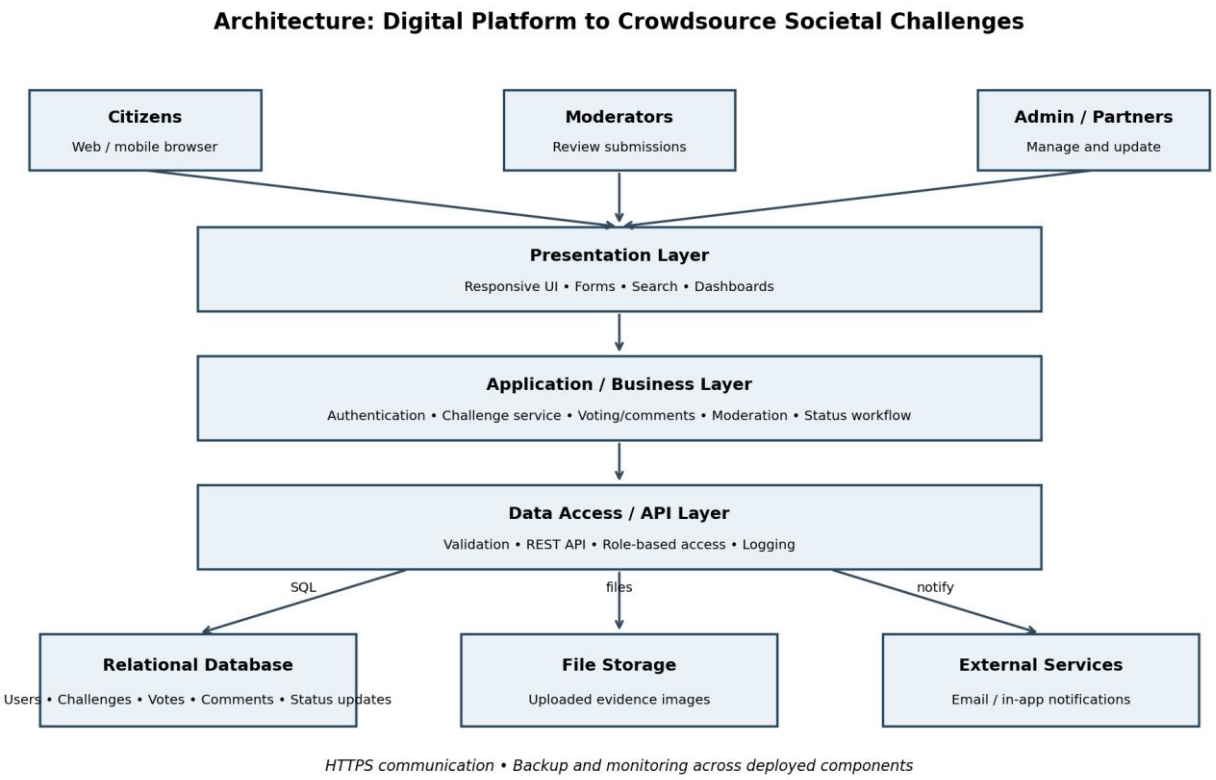
- Separation of concerns: user interface, business rules, and data storage have clear responsibilities.
- Beginner-friendly implementation: the first version can be built and tested as one deployable application with separate modules.
- Maintainability: changes to the interface or business rules can be made without rewriting every part of the system.
- Security: authentication, authorization, validation, and logging can be handled consistently in the application layer.
- Scalability: application instances and storage capacity can be increased as usage grows.
- Future extension: notification, analytics, map, or AI-assisted categorization features can later be extracted into separate services if needed.

2.3 Why Not Start with Microservices?

Microservices can support independent scaling, but they introduce service-to-service communication, deployment, monitoring, and data-consistency complexity. For a student project and an initial version, a layered modular application is simpler to build, test, and maintain. The system can evolve if real usage later requires independent services.

3. Software Architecture Diagram

The diagram shows the main users, application layers, database, file storage, and notification integration.



Communication: users access the platform through HTTPS. The presentation layer sends requests to the application layer. The application layer validates permissions and business rules, then uses the data-access/API layer to read or write information and call external services when required.

4. Explain Components and Their Interactions

Component	Responsibility	Interaction
Presentation Layer	Displays registration, challenge forms, search results, comments, dashboards, and status timelines.	Sends user requests to application functions over HTTPS.
Authentication and Authorization	Verifies login and checks user roles and permissions.	Used before protected actions such as moderation or partner updates.
Challenge Management	Creates, edits, categorizes, searches, and retrieves challenges.	Reads and writes challenge records in the database.
Community Participation	Manages votes, comments, and followed challenges.	Links user actions to the relevant challenge.
Moderation and Reporting	Reviews submissions, handles abuse reports, and flags duplicates.	Records moderation decisions and informs the relevant user.
Status and Partner Updates	Maintains challenge status and progress history.	Authorized partners or admins submit updates; users view the timeline.
Notification Component	Creates email or in-app alerts for key events.	Receives events from the application and uses a configured notification service.
Data Access Layer	Provides controlled database queries and file references.	Connects business functions to database and file storage.

Database	Stores users, challenges, votes, comments, categories, reports, and status updates.	Returns records to the application layer.
File Storage	Stores optional evidence images and attachments.	Stores file references in the database; validates file types and sizes.
Analytics and Administration	Summarizes participation, categories, statuses, and progress.	Uses authorized aggregate data for dashboards and reports.

4.1 System Workflow

1. Citizen registers or logs in to the platform.
2. Citizen submits a challenge with a description, category, area, and optional image.
3. The application validates the submission and saves it in the database with a Submitted status.
4. A moderator reviews the submission and approves it, rejects it, requests clarification, or links it as a duplicate.
5. Other users can search for the challenge, vote, comment, and follow updates.
6. An authorized partner or administrator posts progress and updates the status.
7. The system records the update and sends a notification when notifications are enabled.
8. Administrators and authorized partners view summary reports to understand community needs and progress.

5. Discuss Quality Attributes

Attribute	How the Architecture Supports It
Scalability	The web/application tier can be scaled horizontally; database indexes and query optimization help manage larger datasets.
Security	HTTPS protects data in transit; password hashing, role-based access, validation, and audit logs protect accounts and actions.
Performance	Pagination, indexed search fields, caching of suitable public data, and optimized queries reduce response time.
Reliability	Database transactions, regular backups, input validation, and error handling reduce data loss and inconsistent records.
Maintainability	Layer separation and modular services make it easier to test, debug, and modify individual features.
Availability	Reliable hosting, health monitoring, backups, and documented recovery procedures reduce downtime.

These are design measures and targets; actual performance and availability must be confirmed through testing and deployment monitoring.

6. Justify Architectural Decisions

6.1 Main Decisions

- Use a responsive web interface so citizens can access the platform from computers and mobile devices.
- Use a relational database because users, challenges, votes, comments, and status updates have clear relationships.
- Keep business rules in the application layer to apply validation and permission checks consistently.
- Use role-based access to distinguish citizen, moderator, administrator, and partner capabilities.
- Store uploaded evidence separately from structured database records, while keeping secure file references in the database.
- Use an optional notification integration so the core platform can continue to work if an email provider is unavailable.

6.2 Trade-Offs

Decision	Benefit	Trade-Off
Layered architecture	Clear structure and easier testing	Layer boundaries need discipline to prevent tightly coupled code.
Modular monolith initially	Simple deployment and development	The whole application may need redeployment for some changes.
Relational database	Strong relationships and transactions	Very large-scale search or analytics may later need specialized tools.
External notification service	Avoids building email delivery from scratch	Depends on provider availability and may incur costs.
Moderation before public display	Reduces spam and harmful content	Can delay publication when moderator capacity is limited.

6.3 Future Enhancements

- Add multilingual support for wider community participation.
- Add map-based browsing using approximate public locations where privacy permits.
- Use AI-assisted categorization or duplicate suggestions with human review.
- Integrate with civic or NGO systems through documented APIs where permission is available.
- Introduce a dedicated notification or analytics service if traffic and operational needs justify it.
- Add accessibility testing, stronger monitoring, and disaster-recovery exercises.

Conclusion

The Three-Tier Layered Architecture is a practical starting point for the Digital Platform to Crowdsource Societal Challenges. It separates the user interface, business rules, and data storage while supporting security, maintainability, reliability, and future growth. The architecture diagram and component descriptions show how citizens, moderators, administrators, partner organizations, databases, file storage, and notifications work together. The design should be validated through prototypes, security checks, performance tests, and feedback from intended users.